SOLID Principles

C# Developer Reference — Session 1

Quick Reference

Principle	One-liner	Key question
S — Single Responsibility	One class, one job	How many reasons to change?
O — Open/Closed	Extend, don't modify	Can I add without touching old code?
L — Liskov Substitution	Subclass keeps the promise	Can child replace parent safely?
I — Interface Segregation	Split fat interfaces	Am I forced to implement unused methods?
D — Dependency Inversion	Depend on abstractions	Do I know the concrete type?

S — Single Responsibility Principle

> *"A class should have only one reason to change."*

Each class does exactly one thing. If you need to change email logic, you should only touch EmailService — not UserService, not the repository. Think of a motorbike: a flat tyre only needs the tyre replaced — the engine and brakes are untouched.

■ Violation — 3 reasons to change

```
// ■ BAD — UserService does 3 unrelated jobs

public class UserService

{

    public void CreateUser(string name, string email)

    {

        // 1. Validate

        if (string.IsNullOrEmpty(email))

            throw new Exception("Email cannot be empty");


        // 2. Save to database

        var conn = new SqlConnection("Server=...;Database=...");

        conn.Open();

        var cmd = new SqlCommand("INSERT INTO Users VALUES (@n,@e)", conn);

        cmd.Parameters.AddWithValue("@n", name);

        cmd.Parameters.AddWithValue("@e", email);

        cmd.ExecuteNonQuery();


        // 3. Send welcome email

        var smtp = new SmtpClient("smtp.gmail.com");

        smtp.Send("no-reply@app.com", email, "Welcome!", $"Hi {name}!");

    }
```

```
}
```

■ Correct — one reason to change per class

```
// ■ GOOD — each class has exactly one responsibility

public class UserValidator

{

    public void Validate(string name, string email)

    {

        if (string.IsNullOrEmpty(name))  throw new ArgumentException("Name required");

        if (!email.Contains("@"))        throw new ArgumentException("Invalid email");

    }

}


public class UserRepository

{

    private readonly AppDbContext _db;

    public UserRepository(AppDbContext db) => _db = db;

    public void Save(User user) { _db.Users.Add(user); _db.SaveChanges(); }

}


public class EmailService

{

    public void SendWelcome(string email, string name)

        => Console.WriteLine($"Sending welcome to {email}");

}


// UserService only orchestrates — that is its single responsibility

public class UserService
```

```
{
    private readonly UserValidator _v;
    private readonly UserRepository _r;
    private readonly EmailService _e;
    public UserService(UserValidator v, UserRepository r, EmailService e)
        => (_v, _r, _e) = (v, r, e);

    public void CreateUser(string name, string email)
    {
        _v.Validate(name, email);
        _r.Save(new User { Name = name, Email = email });
        _e.SendWelcome(email, name);
    }
}
```

O — Open/Closed Principle

> *"Open for extension, closed for modification."*

Add new behaviour by writing new code — never by editing code that already works. Like choosing how to get to school: walk, ride a bike, or take a bus. You never rewrite goSchool() — you just pick a different transport that fulfils the same contract.

■ Violation — must modify existing code to extend

```
// ■ BAD — every new payment method requires editing this class

public class PaymentProcessor

{

    public void Process(string type, decimal amount)

    {

        if (type == "CreditCard")

            Console.WriteLine($"Paid {amount} by Credit Card");

        else if (type == "PayPal")

            Console.WriteLine($"Paid {amount} by PayPal");

        // Adding Bitcoin? Must touch this file — risky!

    }

}
```

■ Correct — extend by adding new classes

```
// ■ GOOD — new payment methods never touch PaymentProcessor

public interface IPaymentMethod { void Process(decimal amount); }


public class CreditCardPayment : IPaymentMethod

{

    public void Process(decimal amount) => Console.WriteLine($"Credit Card: {amount}");

}
```

```
public class BitcoinPayment : IPaymentMethod   // added with zero edits to old code
{
    public void Process(decimal amount) => Console.WriteLine($"Bitcoin: {amount}");
}

public class PaymentProcessor   // never changes
{
    public void Process(IPaymentMethod method, decimal amount)
        => method.Process(amount);
}
```

L — Liskov Substitution Principle

> *"A subclass must be substitutable for its parent without breaking the program."*

A child class can override methods, but must still fulfil the promise the parent made. If the parent says Fly() works, every child must fly successfully — throwing an exception breaks the contract.

■ Violation — child cannot honour parent's promise

```
// ■ BAD — Penguin breaks Bird's contract

public class Bird { public virtual void Fly() => Console.WriteLine("Flying"); }


public class Penguin : Bird

{

    public override void Fly()

        => throw new Exception("Penguins cannot fly!"); // ■ runtime crash

}


public void MakeFly(Bird b) => b.Fly();  // blows up when b is Penguin
```

■ Correct — subclass only promises what it can deliver

```
// ■ GOOD — separate what all birds share from what only some birds do

public abstract class Bird { public abstract void Eat(); }


public interface IFlyable { void Fly(); }


public class Sparrow : Bird, IFlyable

{

    public override void Eat() => Console.WriteLine("Sparrow eating");

    public void Fly()          => Console.WriteLine("Sparrow flying");

}


public class Penguin : Bird   // does NOT implement IFlyable — honest!
```

```
{
    public override void Eat() => Console.WriteLine("Penguin eating");
}

// Only types that truly fly are accepted here
public void MakeFly(IFlyable b) => b.Fly();
```

I — Interface Segregation Principle

> *"No class should be forced to implement methods it does not use."*

Keep interfaces small and focused. If an interface is too fat, split it. A Robot should never be forced to implement Eat() or Sleep() just because a Human does those things.

■ Violation — class forced to implement unused methods

```
// ■ BAD — fat interface forces Robot to implement impossible methods

public interface IWorker

{

    void Work();

    void Eat();    // Robot cannot eat

    void Sleep();  // Robot cannot sleep

}


public class Robot : IWorker

{

    public void Work()  => Console.WriteLine("Robot working");

    public void Eat()   => throw new Exception("Robots don't eat!");

    public void Sleep() => throw new Exception("Robots don't sleep!");

}
```

■ Correct — implement only what is relevant

```
// ■ GOOD — small, focused interfaces

public interface IWorkable  { void Work(); }

public interface IEatable   { void Eat();  }

public interface ISleepable { void Sleep(); }


public class Human : IWorkable, IEatable, ISleepable

{
```

```
    public void Work()  => Console.WriteLine("Human working");
    public void Eat()   => Console.WriteLine("Human eating");
    public void Sleep() => Console.WriteLine("Human sleeping");
}

public class Robot : IWorkable   // only what it actually does
{
    public void Work() => Console.WriteLine("Robot working");
}
```

D — Dependency Inversion Principle

> *"High-level modules should not depend on low-level modules. Both should depend on abstractions."*

High-level classes should not know or care about concrete implementations. You only care that you get to school — not whether you walk, ride a bike, or take a bus. Swap the transport any time without touching the high-level logic.

■ Violation — high-level class depends on concrete type

```
// ■ BAD — UserService is hardwired to a concrete EmailService

public class UserService

{

    // Directly creates the dependency — tightly coupled

    private readonly EmailService _email = new EmailService();


    public void CreateUser(string name, string email)

    {

        // ...

        _email.SendWelcome(email, name);  // can never swap or mock this

    }

}
```

■ Correct — depend on interface, inject concrete type from outside

```
// ■ GOOD — depend on the abstraction, not the implementation

public interface IEmailService { void SendWelcome(string email, string name); }


public class SmtpEmailService : IEmailService

{

    public void SendWelcome(string email, string name)

        => Console.WriteLine($"SMTP -> {email}");

}
```

```
public class SendGridEmailService : IEmailService

{

    public void SendWelcome(string email, string name)

        => Console.WriteLine($"SendGrid -> {email}");

}


public class UserService

{

    private readonly IEmailService _email;


    // Dependency is injected — UserService never knows the concrete type

    public UserService(IEmailService email) => _email = email;


    public void CreateUser(string name, string email)

    {

        // ...

        _email.SendWelcome(email, name);

    }

}


// Swap provider any time — zero changes to UserService

var svc = new UserService(new SendGridEmailService());
```

All five principles work together. Applying I early (small interfaces) prevents L violations. D relies on the same interfaces introduced by O. SOLID is not five separate rules — it is one coherent design philosophy.